

End-Semester Examination, May 2025

Game Programming with C++ (CSE 3545) • B.Tech CSE, Semester 6 • Full Solutions

Every sub-question of the 10-question paper is answered below. Each answer leads with the concept in plain language, then the compilable code. Where the question covers something visual (views, sprite sheets, inheritance trees), a diagram is included. Q1 is pure C++; Q2–Q9 are SFML; Q10 is mixed C++ OOP.

Question 1 — Core C++

1(a) Largest digit in an integer

Peel off the last digit with `n % 10`, drop it with `n /= 10`, track the max. Handle negative input with `abs()`.

```
#include <iostream>
#include <cstdlib>
using namespace std;

int main(){
    int n;
    cout << "Enter an integer: ";
    cin >> n;
    n = abs(n);

    int largest = 0;
    if (n == 0) largest = 0;

    while (n > 0) {
        int digit = n % 10;
        if (digit > largest) largest = digit;
        n /= 10;
    }
    cout << "Largest digit = " << largest << endl;
    return 0;
}
```

1(b) Dynamic array via pointer — collect ratings

```
#include <iostream>
using namespace std;

int* getRatings(int n){
    int* arr = new int[n];          // allocate on heap
    for (int i = 0; i < n; i++) {
        cout << "Rating from user " << i+1 << ": ";
        cin >> arr[i];
    }
    return arr;                    // return base pointer
}

int main(){
    int n;
    cout << "Number of users: "; cin >> n;

    int* ratings = getRatings(n);

    cout << "Ratings: ";
    for (int i = 0; i < n; i++) cout << ratings[i] << " ";
    cout << endl;

    delete[] ratings;              // free the memory
    ratings = nullptr;
    return 0;
}
```

Why delete[] not delete? The array form must be paired with the array delete; mixing them is undefined behaviour.

1(c) Billing system with enum + discount function

```
#include <iostream>
using namespace std;

enum ProductCategory { ELECTRONICS, CLOTHING, GROCERIES };

double applyDiscount(double price, ProductCategory cat){
    double pct = 0.0;
    switch (cat) {
        case ELECTRONICS: pct = 0.10; break;    // 10%
        case CLOTHING:    pct = 0.20; break;    // 20%
        case GROCERIES:   pct = 0.05; break;    // 5%
    }
    return price - price * pct;
}

int main(){
    int c; double price;
    cout << "Category (0=ELEC, 1=CLOTHING, 2=GROCERIES): "; cin >> c;
    cout << "Price: "; cin >> price;

    double finalPrice = applyDiscount(price, (ProductCategory)c);
    cout << "Final amount = " << finalPrice << endl;
    return 0;
}
```

Question 2 — SFML basics

2(a) Down-arrow cycles active sprite through array of 6

Use the KeyPressed event (not polling) so one press = one advance. Wrap with modulo to loop back to the top.

```
Sprite sprites[6];           // assume already loaded with textures
int active = 0;              // index of currently-highlighted sprite

Event event;
while (window.pollEvent(event)) {
    if (event.type == Event::KeyPressed) {
        if (event.key.code == Keyboard::Down) {
            active = (active + 1) % 6;    // wrap around
        }
    }
}

// Visual feedback: tint active sprite, others normal
for (int i = 0; i < 6; i++) {
    sprites[i].setColor(i == active ? Color::Yellow : Color::White);
    window.draw(sprites[i]);
}
```

2(b) Time bar — 6-second countdown

Shrink a RectangleShape's width proportional to time left. Use a Clock to track elapsed seconds.

```
const float TIME_LIMIT = 6.0f;           // seconds
RectangleShape timeBar;
timeBar.setFillColor(Color::Red);
timeBar.setPosition(100.f, 50.f);
float fullWidth = 400.f;
timeBar.setSize(Vector2f(fullWidth, 20.f));

Clock clock;

while (window.isOpen()) {
    float elapsed = clock.getElapsedTime().asSeconds();
    float remaining = TIME_LIMIT - elapsed;
    if (remaining < 0) remaining = 0;

    // width shrinks linearly with time remaining
    timeBar.setSize(Vector2f((remaining / TIME_LIMIT) * fullWidth, 20.f));

    window.clear();
    window.draw(timeBar);
    window.display();
}
```

2(c) Score++ on Up-arrow, display with sf::Text

```
Font font;
font.loadFromFile("KOMIKAP_.ttf");

Text scoreText;
scoreText.setFont(font);
scoreText.setCharacterSize(40);
scoreText.setFillColor(Color::White);
scoreText.setPosition(20.f, 20.f);

int score = 0;

Event event;
while (window.pollEvent(event)) {
    if (event.type == Event::KeyPressed) {
        if (event.key.code == Keyboard::Up) {
            score++;
        }
    }
}
```

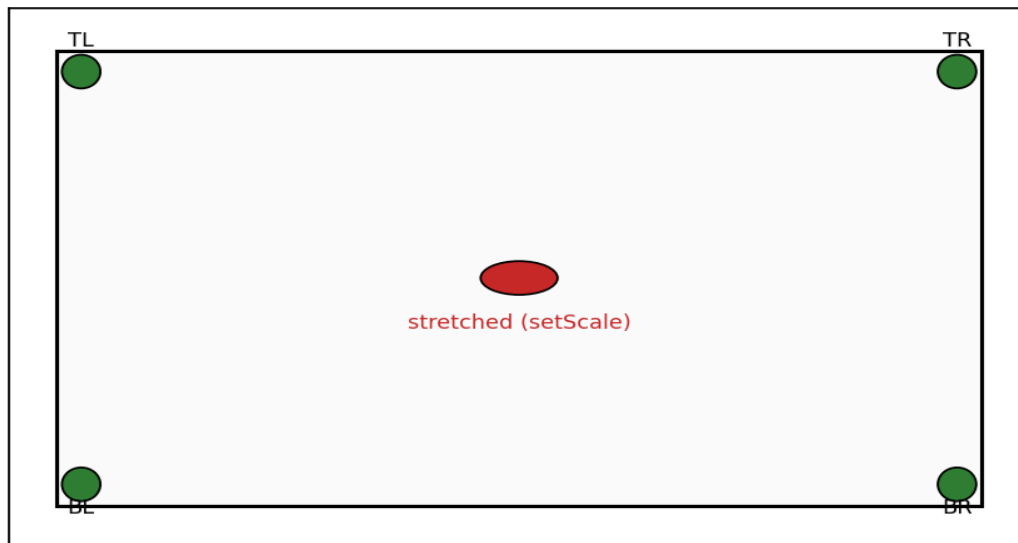
```
}
```

```
scoreText.setString("Score: " + to_string(score));  
window.draw(scoreText);
```

Question 3 — SFML drawing & input

3(a) 4 green corner circles + 1 stretched red centre circle

Q3a: 4 green corner circles + 1 stretched red centre circle



```
const float R = 10.f;           // radius
const float W = 1920.f, H = 1080.f;

// Helper: make one circle at a given centre
auto makeCircle = [](float cx, float cy, Color c){
    CircleShape s(R);
    s.setOrigin(R, R);           // centre the origin
    s.setPosition(cx, cy);
    s.setFillColor(c);
    return s;
};

// 4 green corner circles (a small margin from the edge)
CircleShape tl = makeCircle(20.f, 20.f, Color::Green);
CircleShape tr = makeCircle(W - 20.f, 20.f, Color::Green);
CircleShape bl = makeCircle(20.f, H - 20.f, Color::Green);
CircleShape br = makeCircle(W - 20.f, H - 20.f, Color::Green);

// 1 centre red circle, stretched (e.g. wider horizontally)
CircleShape center = makeCircle(W / 2.f, H / 2.f, Color::Red);
center.setScale(2.f, 1.f);      // stretch horizontally

window.draw(tl); window.draw(tr);
window.draw(bl); window.draw(br);
window.draw(center);
```

3(b) Toggle player between left/right of tree, flip orientation

Two positions: just-left and just-right of the tree. On Left arrow, place to the left and face right (no flip). On Right arrow, place to the right and face left (mirror via `setScale(-1, 1)`).

```
const float TREE_X = 960.f;      // tree centre
const float TREE_Y = 540.f;
const float OFFSET = 80.f;

Sprite player; player.setTexture(playerTex);
player.setOrigin(playerTex.getSize().x / 2.f,
                 playerTex.getSize().y / 2.f);

Event event;
while (window.pollEvent(event)) {
    if (event.type == Event::KeyPressed) {
        if (event.key.code == Keyboard::Left) {
            player.setPosition(TREE_X - OFFSET, TREE_Y);
```

```

        player.setScale( 1.f, 1.f);    // face right (toward tree)
    }
    if (event.key.code == Keyboard::Right) {
        player.setPosition(TREE_X + OFFSET, TREE_Y);
        player.setScale(-1.f, 1.f);    // flipped, face left toward tree
    }
}
window.draw(player);

```

3(c) Play death.wav on player–branch collision

```

SoundBuffer deathBuffer;
deathBuffer.loadFromFile("death.wav");
Sound deathSound;
deathSound.setBuffer(deathBuffer);

// Inside the game loop, after updating positions:
if (player.getGlobalBounds().intersects(branch.getGlobalBounds())) {
    deathSound.play();
    // also: state = State::GAME_OVER;
}

```

Question 4 — Classes & encapsulation

4(a) MyBat class with 4 ints + setData/getData

```
class MyBat {
private:
    int a, b, c, d;           // four int data members
public:
    void setData(int w, int x, int y, int z){
        a = w; b = x; c = y; d = z;
    }
    void getData(){
        cout << "a=" << a << " b=" << b
            << " c=" << c << " d=" << d << endl;
    }
};
```

4(b) Draw a 10×10 rectangle in a 1920×1080 window

```
class MyBox {
private:
    RectangleShape m_Shape;
public:
    MyBox(){
        m_Shape.setSize(Vector2f(10.f, 10.f));
        m_Shape.setFillColor(Color::White);
        m_Shape.setPosition(960.f, 540.f);    // centre of 1920x1080
    }
    RectangleShape getShape() const { return m_Shape; }
};

int main(){
    RenderWindow window(VideoMode(1920, 1080), "MyBox");
    MyBox box;
    while (window.isOpen()) {
        Event e;
        while (window.pollEvent(e))
            if (e.type == Event::Closed) window.close();
        window.clear();
        window.draw(box.getShape());
        window.display();
    }
}
```

4(c) Bat class — 100×5 bat at a parameterised position

```
class Bat {
private:
    RectangleShape m_Sprite;
public:
    Bat(float x, float y){
        m_Sprite.setSize(Vector2f(100.f, 5.f));
        m_Sprite.setFillColor(Color::White);
        m_Sprite.setPosition(x, y);
    }
    RectangleShape getShape() const { return m_Sprite; }
};

int main(){
    RenderWindow window(VideoMode(1920, 1080), "Bat");
    Bat bat(910.f, 1000.f);    // near bottom-centre
    while (window.isOpen()) {
        Event e;
        while (window.pollEvent(e))
            if (e.type == Event::Closed) window.close();
        window.clear();
        window.draw(bat.getShape());
        window.display();
    }
}
```


Question 5 — OOP: constructors, overloading, inheritance

5(a) Temperature: F → C in parameterised constructor

```
class Temperature {
private:
    double celsius;
public:
    Temperature(double fahrenheit){
        celsius = (fahrenheit - 32.0) * 5.0 / 9.0;
    }
    double getCelsius() const { return celsius; }
};

int main(){
    Temperature t(98.6);                // body temperature in F
    cout << "Celsius = " << t.getCelsius(); // ≈ 37
}
```

5(b) AreaCalculator with overloaded calculate()

```
class AreaCalculator {
public:
    // square – one parameter
    double calculate(double side){
        return side * side;
    }
    // rectangle – two parameters
    double calculate(double length, double breadth){
        return length * breadth;
    }
};

int main(){
    AreaCalculator a;
    cout << "Square : " << a.calculate(5) << endl; // 25
    cout << "Rectangle: " << a.calculate(4, 6) << endl; // 24
}
```

Function overloading is resolved at *compile time* based on the number/types of arguments. The two functions are distinct entities even though they share a name.

5(c) Person → Student inheritance with overridden display()

```
#include <iostream>
#include <string>
using namespace std;

class Person {
protected:
    string name;
    int age;
public:
    Person(string n, int a) : name(n), age(a) {}
    void display(){
        cout << "Name: " << name << ", Age: " << age << endl;
    }
};

class Student : public Person {
private:
    int rollNumber;
public:
    Student(string n, int a, int r)
        : Person(n, a), rollNumber(r) {} // init base via init-list

    void display(){ // override
        cout << "Name: " << name
            << ", Age: " << age
            << ", Roll Number: " << rollNumber << endl;
    }
};
```

```
        << ", Roll: " << rollNumber << endl;
    }
};

int main(){
    Student s("Ritu", 21, 101);
    s.display();        // uses Student's version
}
```

Question 6 — Cursor, views, game states

6(a) Custom crosshair following the mouse, default cursor hidden

```
window.setMouseCursorVisible(false);

Texture crossTex;
crossTex.loadFromFile("crosshair.png");
Sprite crosshair(crossTex);

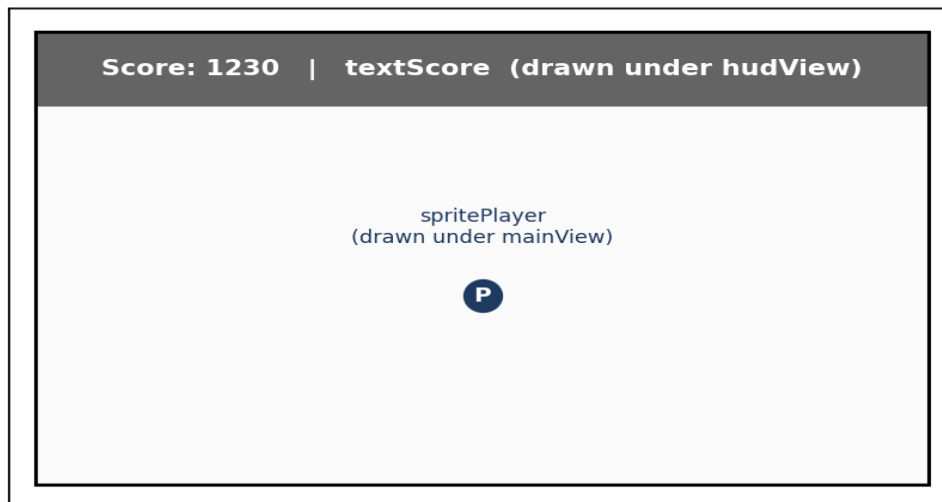
// Centre the origin so the crosshair is centred on the cursor
crosshair.setOrigin(crossTex.getSize().x / 2.f,
                   crossTex.getSize().y / 2.f);

// Inside the game loop:
Vector2i mousePos = Mouse::getPosition(window);
crosshair.setPosition(window.mapPixelToCoords(mousePos));

window.draw(crosshair);
```

6(b) Two views: mainView (world) + hudView (overlay)

Q6b: mainView for world, hudView for score overlay



```
// --- draw the world ---
window.setView(mainView);
window.draw(spritePlayer);

// --- draw the HUD on top ---
window.setView(hudView);
window.draw(textScore);

window.display();
```

Why two views? The world view scrolls with the player. The HUD view doesn't — it's fixed relative to the window so the score stays in the same screen corner regardless of camera movement.

6(c) Enum-class GameState toggled by X key

```
enum class GameState { GameOver, Playing };

Texture texLanding, texPlay;
texLanding.loadFromFile("gameLanding.png");
texPlay.loadFromFile("gamePlay.png");

Sprite background;
GameState state = GameState::GameOver;
background.setTexture(texLanding);

Event event;
while (window.pollEvent(event)) {
    if (event.type == Event::KeyPressed
```

```
        && event.key.code == Keyboard::X) {

        if (state == GameState::GameOver) {
            state = GameState::Playing;
            background.setTexture(texPlay);
        } else {
            state = GameState::GameOver;
            background.setTexture(texLanding);
        }
    }

}

window.clear();
window.draw(background);
window.display();
```

Question 7 — Player class, rotation, camera follow

7(a) Player::spawn(View hudView) — start at arena centre

The signature in the question takes a view; we read its size to find the centre. (In the Zombie Arena codebase the more usual signature takes an `IntRect` — either is acceptable.)

```
class Player {
private:
    Sprite    m_Sprite;
    Texture   m_Texture;
    Vector2f  m_Position;
public:
    Player(){
        m_Texture.loadFromFile("player.png");
        m_Sprite.setTexture(m_Texture);
        m_Sprite.setOrigin(25.f, 25.f);
    }

    void spawn(View hudView){
        Vector2f viewSize = hudView.getSize();
        m_Position.x = viewSize.x / 2.f;           // centre of the view
        m_Position.y = viewSize.y / 2.f;
        m_Sprite.setPosition(m_Position);
    }
    Sprite getSprite() const { return m_Sprite; }
};
```

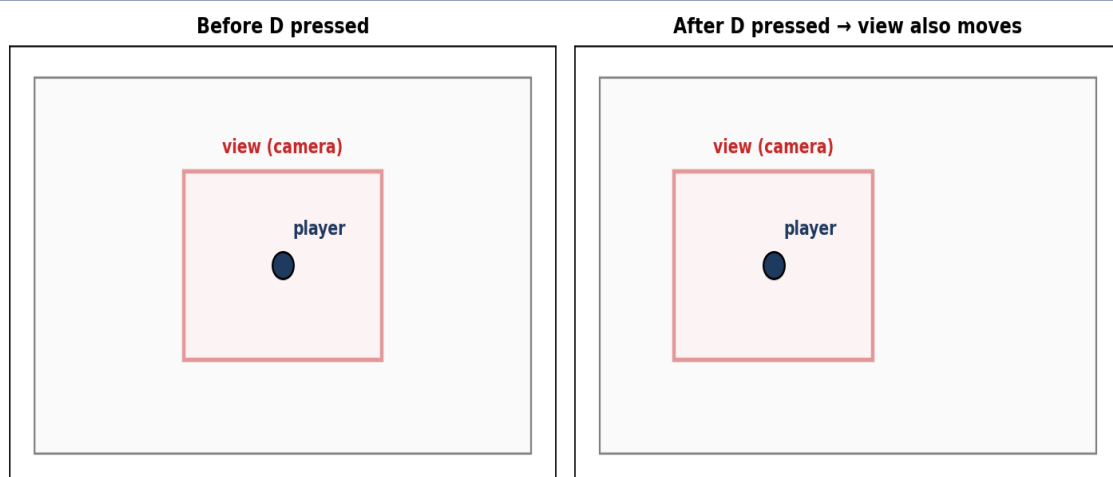
7(b) Player::rotatePlayer(Vector2i mouse) — face the cursor

```
#include <cmath>
#define PI 3.14159f

void Player::rotatePlayer(Vector2i mousePosition){
    float dx = mousePosition.x - m_Sprite.getPosition().x;
    float dy = mousePosition.y - m_Sprite.getPosition().y;

    // atan2 → radians; convert to degrees for SFML
    float angle = atan2f(dy, dx) * 180.f / PI;
    m_Sprite.setRotation(angle);
}
```

7(c) D key moves player; camera follows so player stays centred



```
View mainView(FloatRect(0, 0, 1920, 1080));

Vector2f playerPos(960.f, 540.f); // start at centre
Sprite player;                     // assume textured

float speed = 200.f;               // px/sec
Clock clock;
```

```

while (window.isOpen()) {
    float dt = clock.restart().asSeconds();

    // --- D key moves player LEFT (as in the question) ---
    if (Keyboard::isKeyPressed(Keyboard::D)) {
        playerPos.x -= speed * dt;
    }
    player.setPosition(playerPos);

    // --- camera follows the player ---
    mainView.setCenter(playerPos);

    window.setView(mainView);
    window.clear();
    // ... draw background ...
    window.draw(player);
    window.display();
}

```

Trick of camera-follow: the player's world position keeps changing, but because the view's centre tracks that exact position, the player *appears* stationary on screen and the world appears to scroll past.

Question 8 — Angles, movement, spawning

8(a) Angle from BLOATER to player + setRotation

```
#include <cmath>
#define PI 3.14159f

// (x1, y1) = player, (x2, y2) = BLOATER position
float dx = x1 - x2;
float dy = y1 - y2;

float angle = atan2f(dy, dx) * 180.f / PI;
bloaterSprite.setRotation(angle); // bloater now faces the player
```

8(b) Move zombie toward player

```
// Direction vector from zombie to player
float dx = x1 - x2;
float dy = y1 - y2;
float length = sqrtf(dx * dx + dy * dy);

if (length > 0) { // avoid divide-by-zero
    dx /= length; // normalise
    dy /= length;
}

float speed = 75.f; // px/sec
x2 += dx * speed * elapsedTime;
y2 += dy * speed * elapsedTime;

zombieSprite.setPosition(x2, y2);
```

Why normalise? Without dividing by length, the zombie would move faster when the player is far away (longer vector) and slower when close. Normalising fixes the speed at exactly speed px/sec regardless of distance.

8(c) Spawn 5 zombies at random positions inside an arena

```
#include <cstdlib>
#include <ctime>

class Zombie {
public:
    void spawn(float x, float y){ /* ... sets m_Position, sprite ... */ }
};

const int NUM = 5;
IntRect arena(0, 0, 1000, 800);

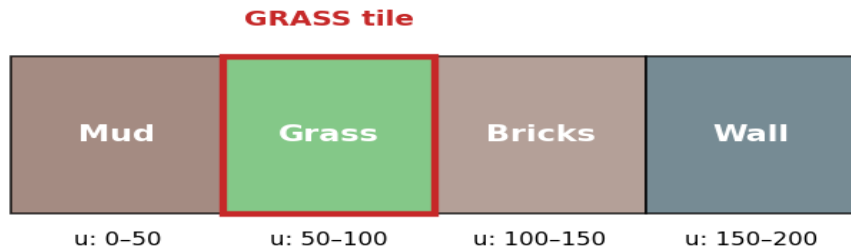
Zombie zombies[NUM];
srand((unsigned)time(0));

for (int i = 0; i < NUM; i++) {
    // Random x within arena bounds (avoid the wall border, e.g. 50 px)
    float x = (rand() % (arena.width - 100)) + 50;
    float y = (rand() % (arena.height - 100)) + 50;
    zombies[i].spawn(x, y);
}
```

Question 9 — Sprite sheets, bullets, VertexArray

9(a) Draw a single GRASS tile at screen (100, 100)

Horizontal sprite sheet (background_sheet.png)



In the sheet shown, the tiles are arranged **horizontally**: Mud (0–50), Grass (50–100), Bricks (100–150), Wall (150–200), all with v running 0–50. We use an IntRect to grab just the grass slice.

```
Texture sheet;
sheet.loadFromFile("background_sheet.png");

Sprite grass;
grass.setTexture(sheet);
// Pick the grass tile out of the sheet (left=50, top=0, w=50, h=50)
grass.setTextureRect(IntRect(50, 0, 50, 50));
grass.setPosition(100.f, 100.f);    // screen position

window.draw(grass);
```

9(b) Fire a 2×2 bullet from player toward zombie

```
RectangleShape bullet;
bullet.setSize(Vector2f(2.f, 2.f));
bullet.setFillColor(Color::Yellow);
bullet.setPosition(x1, y1);    // start at player

// Direction to zombie
float dx = x2 - x1;
float dy = y2 - y1;
float len = sqrtf(dx * dx + dy * dy);
if (len > 0) { dx /= len; dy /= len; }

float speed = 1000.f;    // px/sec
Vector2f velocity(dx * speed, dy * speed);

// In the game loop:
bullet.move(velocity * elapsedTime);
window.draw(bullet);
```

9(c) Resize arena 500×500 → 600×600, keep 50×50 tiles

500×500 arena had 10×10 = 100 tiles. New 600×600 with 50-px tiles → 12×12 = 144 tiles. Allocate fresh VertexArray and rewrite.

```
const int TILE = 50;
IntRect arena(0, 0, 600, 600);    // new size

int cols = arena.width / TILE;    // 12
int rows = arena.height / TILE;    // 12

VertexArray rVA;
rVA.setPrimitiveType(Quads);
rVA.resize(cols * rows * 4);    // 12 x 12 x 4 = 576 vertices

int v = 0;
for (int w = 0; w < cols; w++) {
    for (int h = 0; h < rows; h++) {
```



```

    rVA[v + 0].position = Vector2f(w * TILE,      h * TILE);
    rVA[v + 1].position = Vector2f((w+1) * TILE,  h * TILE);
    rVA[v + 2].position = Vector2f((w+1) * TILE,  (h+1) * TILE);
    rVA[v + 3].position = Vector2f(w * TILE,      (h+1) * TILE);

    // pick texture as needed (mud / grass / wall) – omitted for brevity
    v += 4;
}
}

```

Question 10 — Centred text, multiple inheritance, operator overload

10(a) Centred "Press Enter to Start" → "Playing..." on Enter

Trick to keep text centred even as its content changes: call `setOrigin` to the centre of the text's local bounds **every time** the string changes, then set position to the window centre.

```
Font font;
font.loadFromFile("KOMIKAP_.ttf");

Text msg;
msg.setFont(font);
msg.setCharacterSize(50);
msg.setFillColor(Color::White);
msg.setString("Press Enter to Start");

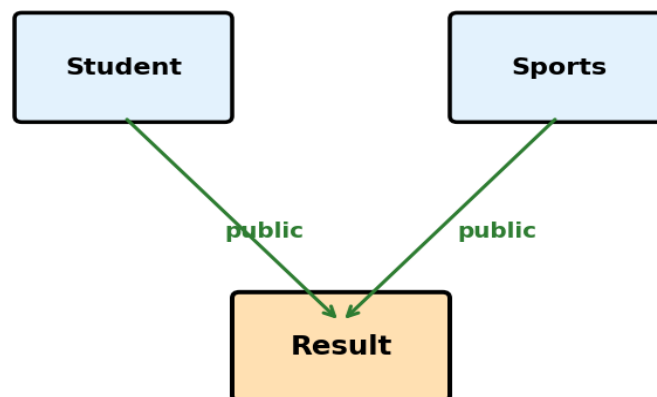
const float WIN_W = 1920.f, WIN_H = 1080.f;

// Helper lambda – re-centres the text whenever the string changes
auto centerText = [&]() {
    FloatRect b = msg.getLocalBounds();
    msg.setOrigin(b.left + b.width / 2.f,
                  b.top + b.height / 2.f);
    msg.setPosition(WIN_W / 2.f, WIN_H / 2.f);
};
centerText();

Event event;
while (window.pollEvent(event)) {
    if (event.type == Event::KeyPressed
        && event.key.code == Keyboard::Return) {
        msg.setString("Playing...");
        centerText(); // re-centre with new bounds
    }
}
window.draw(msg);
```

10(b) Result inherits from Student + Sports (multiple inheritance)

Q10b: Result inherits from Student and Sports



```
#include <iostream>
using namespace std;

class Student {
protected:
    int marks;
public:
    Student(int m) : marks(m) {
        cout << "Student constructor" << endl;
    }
};
```

```

class Sports {
protected:
    int score;
public:
    Sports(int s) : score(s) {
        cout << "Sports constructor" << endl;
    }
};

class Result : public Student, public Sports {
private:
    int total;
public:
    Result(int m, int s) : Student(m), Sports(s) {
        total = marks + score;
        cout << "Result constructor – total = " << total << endl;
    }
};

int main(){
    Result r(80, 15);
    return 0;
}

```

Output:

```

Student constructor
Sports constructor
Result constructor – total = 95

```

Base constructors fire in the order written in the derivation list (*Student* then *Sports*), then the derived constructor. Destructors would run in the reverse order.

10(c) Complex class with operator+ overloaded as a member

```

#include <iostream>
using namespace std;

class Complex {
private:
    double real, imag;
public:
    Complex(double r = 0, double i = 0) : real(r), imag(i) {}

    // Member-function operator overload
    Complex operator+(const Complex& other) const {
        return Complex(real + other.real,
                        imag + other.imag);
    }

    void display() const {
        cout << real << " + " << imag << "i" << endl;
    }
};

int main(){
    Complex a(3, 4);
    Complex b(1, 2);
    Complex c = a + b;           // calls a.operator+(b)
    c.display();                 // prints: 4 + 6i
}

```

Member vs friend? As a member, `a + b` means `a.operator+(b)` — the left operand must be a `Complex`. If you wanted mixed types like `5 + b`, you'd need a friend / non-member version.

Quick reference — patterns that appear repeatedly

1. Angle to face a target

```
float angle = atan2f(target.y - me.y, target.x - me.x) * 180.f / PI;
sprite.setRotation(angle);
```

2. Move toward a target at fixed speed

```
float dx = tx - mx, dy = ty - my;
float len = sqrtf(dx*dx + dy*dy);
if (len > 0) { dx /= len; dy /= len; }
mx += dx * speed * dt;
my += dy * speed * dt;
```

3. Centre a Text object (works after every setString)

```
FloatRect b = text.getLocalBounds();
text.setOrigin(b.left + b.width / 2, b.top + b.height / 2);
text.setPosition(winW / 2, winH / 2);
```

4. Pick one tile from a sprite sheet

```
sprite.setTexture(sheet);
sprite.setTextureRect(IntRect(uLeft, vTop, tileW, tileH));
```

5. Camera follows the player

```
mainView.setCenter(player.getPosition());
window.setView(mainView);
```

6. AABB collision test

```
if (a.getGlobalBounds().intersects(b.getGlobalBounds())) { /* hit */ }
```

7. Random in [a, b]

```
int n = (rand() % (b - a + 1)) + a;    // seed once: srand(time(0));
```

8. Construction order — multiple inheritance

Base constructors fire **left-to-right** in the derivation list, then the derived constructor. Destructors fire in reverse.

9. Dynamic memory

```
int* arr = new int[n];
// ... use arr ...
delete[] arr;          // ALWAYS pair new[] with delete[]
arr = nullptr;
```

10. Operator overload signatures

Member: `T operator+(const T& other) const`; — left operand must be your class.

Friend / non-member: `T operator+(const T& a, const T& b)`; — supports mixed-type left operand.

Exam strategy: every SFML question follows the same skeleton — (1) load resources, (2) handle events, (3) update positions/state, (4) draw under the right view, (5) display. Memorise the skeleton and the per-question logic slots in cleanly.